

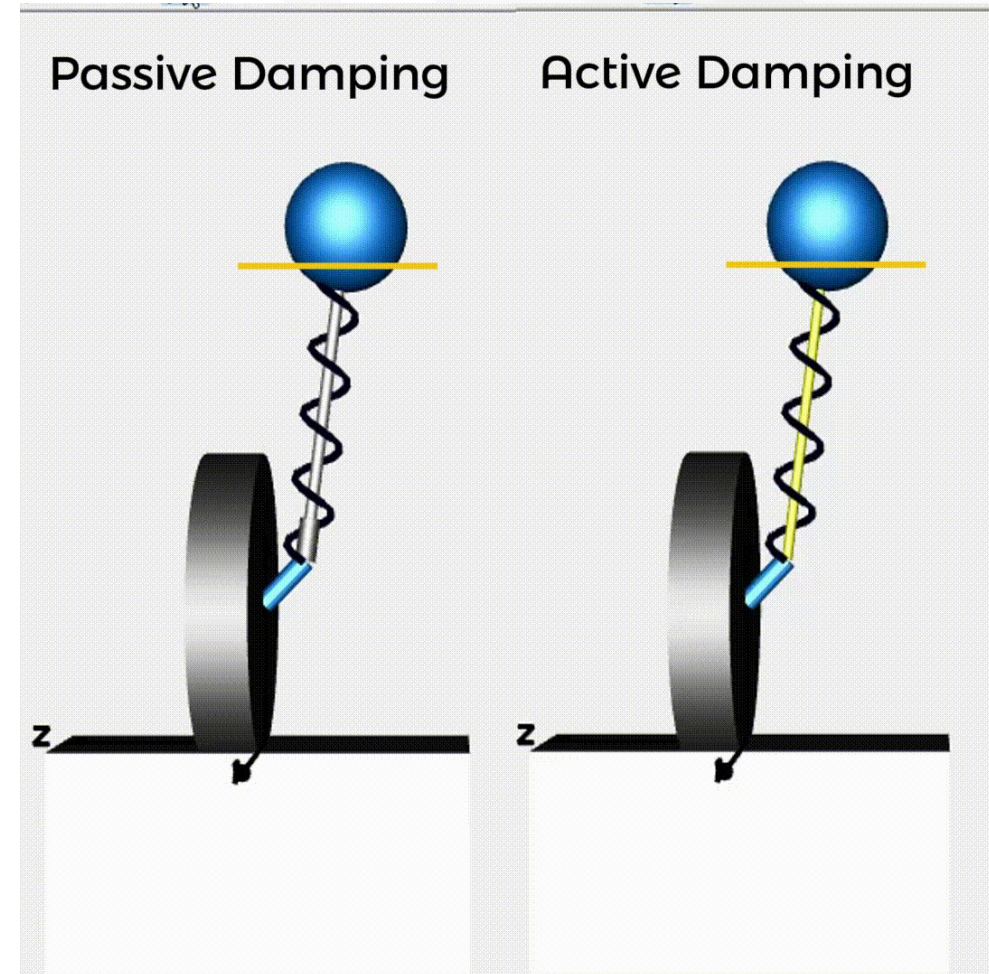
Optimization-Driven Analysis with Physical Models

A Modelica and FMUs Hands-On

Marco Limonciello · Politecnico di Milano · 3 June 2026

Where this started

- Politecnico di Milano (with Prof. Ferretti)
 - Simulation Techniques and Tools - Course
 - Modelica
 - Parameter Estimation - Thesis
 - Modelica, FMI, Python
- Personal Project
 - Semi-Active Damping Suspension



From Physics to Decisions

- We have a **physical system**
- We want to **answer questions** about it
- So we: **Model** it → **Pose a question** → **Solve for an answer**

Simulation

"What happens if I do X?"

Optimisation

"What's the best X I could do?"

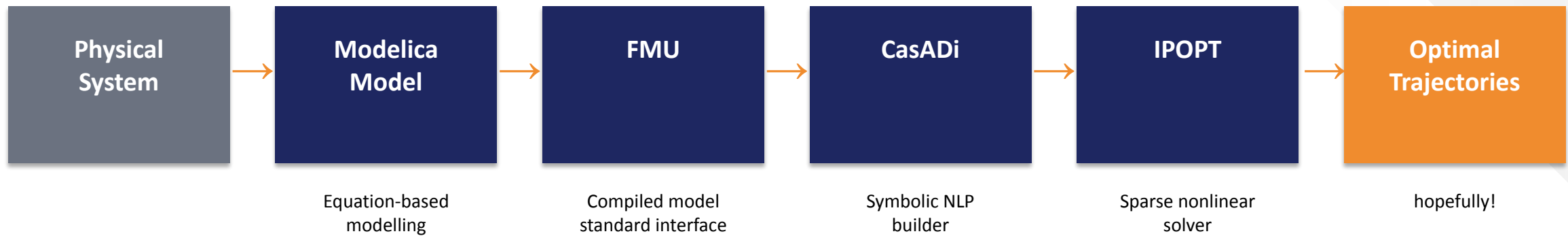
The Building Blocks

- **Model** describes the physics
given states and an inputs, how does the system evolve?
- **OCP** defines the question
objective, constraints, horizon
- **Solver** finds the answer
searches the space of possible inputs for the optimal one

The Example

- **Model:** A battery under charge
State of Charge (SoC) and Temperature evolution
- **OCP:**
 - Objective: Minimise charging cost over electricity prices
 - Subject to thermal and charge constraints

The Pipeline



Physical System

- A battery under charge
- Charging generates heat - electrical losses inside the cell
- Heat flows from the core, through the surface, into the air
- A cooling system activates above a threshold temperature

Modelica Model - Main Equations

- **Electrical :**
 - $I = P_{\text{charge}} / V_{\text{term}}$
 - $V_{\text{term}} = V_{\text{oc}} - I * R_{\text{ohm}}$
- **Thermal:**
 - $Q_{\text{gen}} = I^2 * R_{\text{ohm}}$
 - $\dot{\theta} = Q_{\text{cond}} - Q_{\text{conv}} - Q_{\text{cool}}$
 - $C_{\text{batt}} * \text{der}(T_{\text{batt}}) = Q_{\text{gen}} - Q_{\text{cond}}$
- **Active Cooling:**
 - if $T_{\text{batt}} > T_{\text{cool_on}}$ then $Q_{\text{cool}} = Q_{\text{cool_max}}$ else 0
- **Charge:**
 - $\text{der}(\text{SoC}) = P_{\text{charge}} / E_{\text{cap}}$



POLITECNICO
MILANO 1863

OpenModelica



FMU Export/Import

Export: OpenModelica compiles $\rightarrow$ FMU

(.fmu = zip with binary + `modelDescription.xml`)

Import: The FMU becomes a callable function in Python

`f(x, u, p)  $\rightarrow$   $\dot{x}$`

- States, controls, parameters in - derivatives out
- Parameters tuneable from Python without re-export

Posing the Problem

minimise $\sum \text{price}(k) \times P_{\text{charge}}(k) \times \Delta t$ $\leftarrow$ electricity cost

subject to $\dot{x} = f(x, u, p)$ $\leftarrow$ model dynamics

$T_{\text{batt}} \leq T_{\text{max}}$ $\leftarrow$ safety

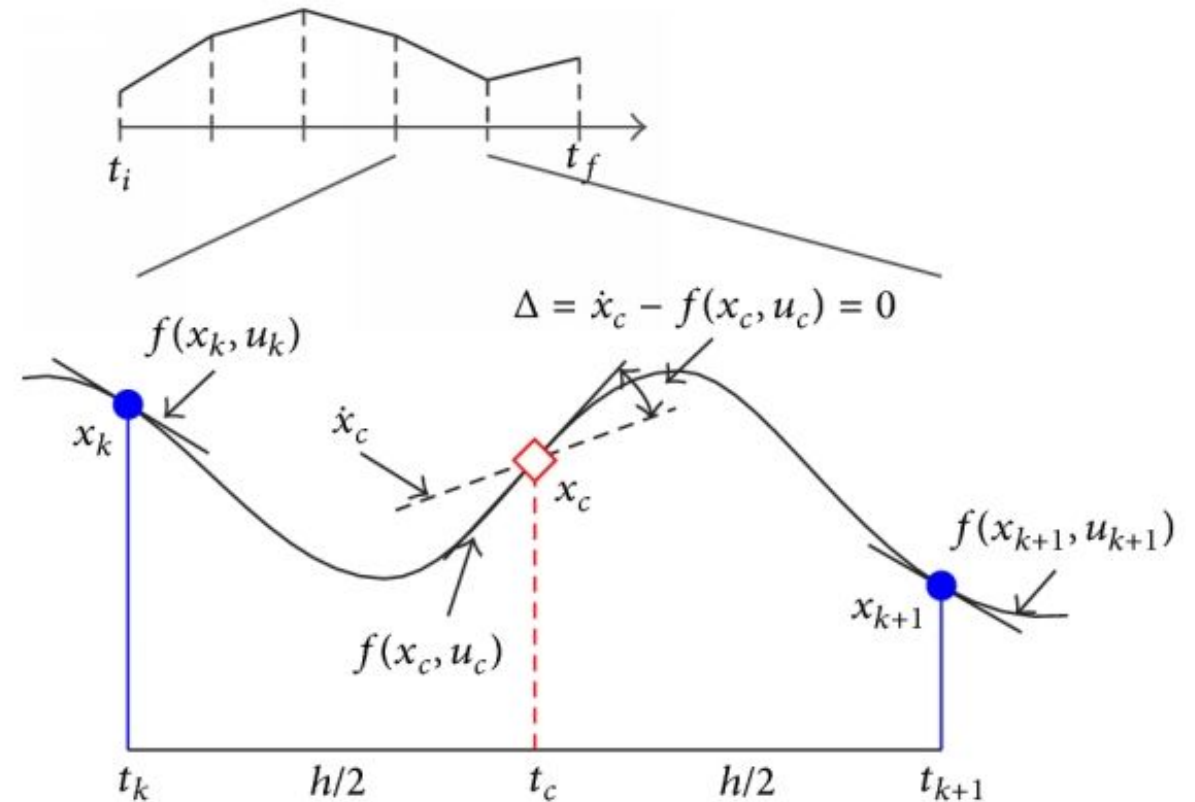
$0 \leq P_{\text{charge}} \leq P_{\text{max}}$ $\leftarrow$ hardware

$\text{SoC}(\text{end}) \geq \text{SoC}_{\text{target}}$ $\leftarrow$ must charge enough

- Price signal diagram: three zones (medium $\rightarrow$ cheap $\rightarrow$ expensive)

Direct Collocation

- discretise into N intervals
- state values at every point are decision variables
- dynamics $\rightarrow$ equality constraints





POLITECNICO
MILANO 1863

CasADi



What Went Wrong

The solver is NOT broken - it's doing its best with bad information

Two categories of problems:

Structural: the model has hidden complexity the solver can't see
(electrical nonlinear loop)

Smoothness: discrete switches destroy gradient information
(if-else thermostat)

(some considerations ...)

NL Loops: BLT Structure

the compiler sorts your equations into sequential blocks

Assignments

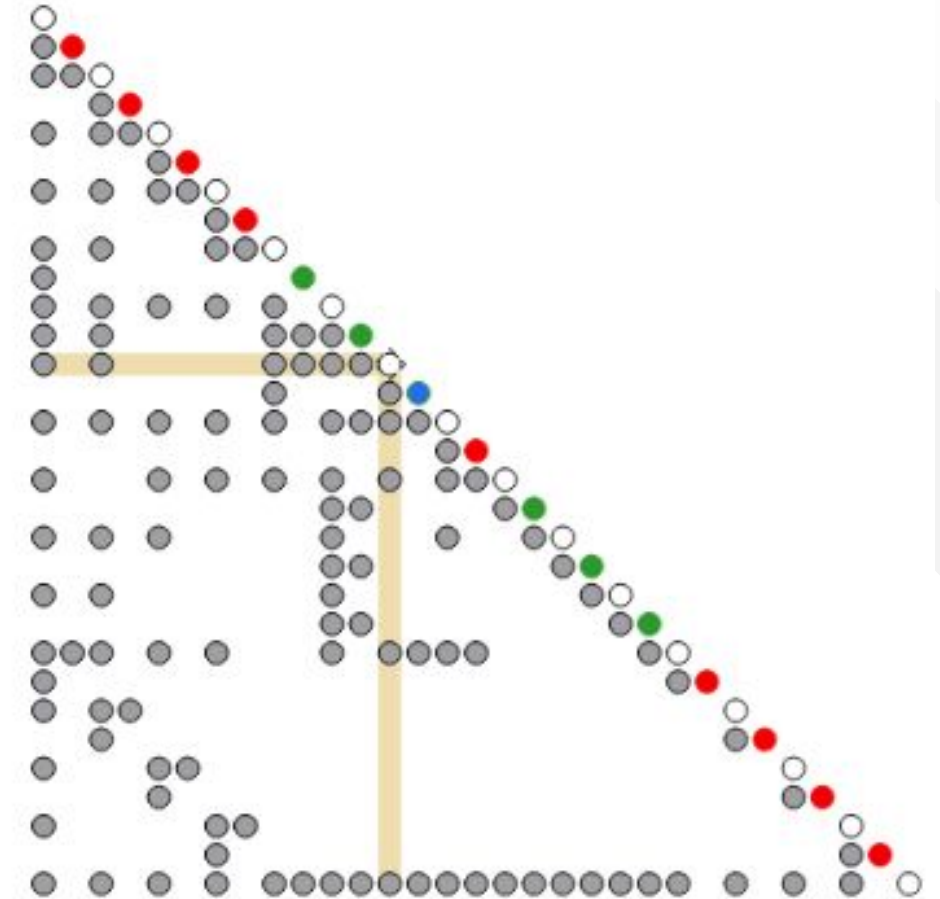
one equation, one unknown, direct solve

Linear loops

system of linear equations,
direct methods (LU factorisation)

Nonlinear loops

mutual nonlinear dependence,
needs iterative solve (Newton)





NL Loops: How to Deal with

Methods exists with their Pros and Cons, here's some:

- **Model Reformulation**

Inputs choice, do algebra yourself ...

- **Transient State Addition**

The algebraic variable becomes a state with fast dynamics

- **Delay Decoupling**

break the algebraic coupling by using the previous time-step value

- **Expose the loop to the solver**

don't hide it in the FMU, let solver handle it

- ...

NL Loops: Our Model

BLT decomposition sorts equations into assignments $\rightarrow$ linear $\rightarrow$ nonlinear blocks.
Our nonlinear loop is the electrical one ($I \leftrightarrow V_term$)

But this loop is analytically solvable:

- Substitute: $V^2 - V_{oc} \cdot V + P \cdot R = 0$
- Under the hood:
 - Quadratic $\rightarrow$ closed-form solution with a caveat

After substitution:

- Loop disappears
- Temp variables appear
- Only direct assignments

*Structural reformulation can be cleaner than any numerical workaround.
The compiler's BLT is mechanical - it doesn't necessarily recognise solvable patterns.*

NL Loops - Fix





Discontinuity Problem

if $T_{\text{batt}} > T_{\text{cool_on}}$ then $Q_{\text{cool}} = Q_{\text{cool_max}}$ else 0

- $\partial Q_{\text{cool}} / \partial T_{\text{batt}} = 0$ on both sides
- Collocation evaluates $f(x, u, p)$ at discrete points

Fix: algebraic smoothing, replace if-else with tanh:

$$Q_{\text{cool}} = Q_{\text{cool_max}} * 0.5 * (1 + \tanh(k * (T_{\text{batt}} - T_{\text{cool_on}})))$$

- Smooth everywhere, differentiable, gradient flows through
- Steepness k trade-off

Other Improvements

Analytic Jacobians - derivatives computed analytically with automatic differentiation

- Without: multiple evaluation FMU → finite differences (noisy, slow)
- With: one call per direction → exact, fast, no noise
- **CONDITIONS:** Continuity, Differentiability, No discrete events

Scaling - IPOPT is sensitive to variable magnitudes; scale states/controls to $O(1)$

Mesh Refinement - more collocation intervals where dynamics are fast, fewer where slow

Let's try again



Discussion

- Best practices:
 - Ensure model is well behaved before tuning OCP
 - Match model complexity to the context
 - Remove discrete dynamics
 - Expose structure to the solver - don't hide it in black boxes
- What else can this OCP can answer?
 - Parameter sensitivities
 $\partial(\text{optimal cost})/\partial p$ - "What happens to cost if R_0 increases ?"

One Model, Multiple Contexts

	Real-time	Simulation	Optimal Control
Uses	HIL, real-time inference	Model validation, design exploration	Trajectory optimisation, NMPC
Tension	Deterministic (bounded) execution time	Catch all dynamics faithfully	Easy to break Difficult to pose
Considerations	Newton loops (non-deterministic).	Most forgiving. Events, stiffness, NL loops	AD critical. Events smoothed. expose NL loops

(...)